

PROYECTO INTEGRADOR · ÁREA MECATRÓNICA



AETHER

INDUSTRIAL SYSTEMS

INFORME DE IMPLEMENTACIÓN INDUSTRIAL DEL PROYECTO INTEGRADOR EN EL ÁREA MECATRÓNICA

INTEGRANTES

Mario Iván Méndez Álvarez
Sadher Isaac Torres Galindo
Pedro José Pallares Muñoz
David Gerardo Garnica Chaparro
Juan Emmanuel Martínez Aguirre
Uriel Armando Rivero Rivero

Grupo: 9IME1M

Universidad Tecnológica de Chihuahua Sur
Profesor encargado: Jesús Fernando Rivera Soto

CHIHUAHUA, MÉXICO · 2026

Profesor encargado del proyecto: Jesús Fernando Rivera Soto

Nombre asumido a partir del formato de estadías previo del mismo equipo (SPECTRABOT); confirmar antes de entrega.

ABSTRACT

Problem Statement

Aether is an academic integrator project that builds a complete automotive manufacturing line combining two industrial robotic arms, a binary vision verification system and an autonomous mobile inventory unit. A KUKA KR6 places automotive parts on a conveyor, a UR5 assembles them through pre-programmed trajectories, a vision system confirms the assembly outcome and a mobile robot, Aether Inventory, collects and transports finished parts while keeping a traceable record of inventory.

The system combines hybrid perception (YOLO detection with barcode and QR identification), ArUco-based position verification, edge computing on an NVIDIA Jetson Orin Nano Super and a peer-to-peer handshake protocol between stations instead of a central orchestrator. At the time of this report the perception pipeline, inventory engine, backend and dashboard run on real hardware, while robotic manipulation, autonomous navigation and the physical manufacturing line remain pending the arrival of the robotic arms and the mobile robot chassis.

The project follows a strict honesty principle throughout its data schema and its documentation: no capability or measurement is reported unless it has been verified against real hardware.

ÍNDICE GENERAL

ABSTRACT	3
ÍNDICE GENERAL	4
ÍNDICE DE FIGURAS, TABLAS Y FRAGMENTOS DE CÓDIGO	5
INTRODUCCIÓN	7
CAPÍTULO 1. LA IDEA DEL PROYECTO	8
1.1 Contexto	8
1.2 Antecedentes del problema	8
1.3 Objetivo del proyecto	8
CAPÍTULO 2. DISEÑO DEL PROYECTO	11
2.1 Marco de decisiones de Fase 0	11
2.2 Componentes del sistema	12
2.3 Diagnóstico	13
2.4 Alternativas de solución	13
CAPÍTULO 3. EJECUCIÓN DEL TRABAJO	15
3.1 Arquitectura general	15
3.2 Estructura de carpetas	15
3.3 Percepción	21
3.4 Localización	22
3.5 Navegación	23
3.6 Inventory Engine	23
3.7 Backend y comunicaciones	24
3.8 Dashboard	26
3.9 Línea de manufactura	28
CAPÍTULO 4. ENTORNO VIRTUAL / FÍSICO Y MANTENIMIENTO	31
4.1 Entorno de desarrollo actual	31
4.2 Entorno físico planeado	31
4.3 Mantenimiento	31
4.4 Manual de seguridad	32
CONCLUSIONES Y RECOMENDACIONES	33
REFERENCIAS	34

ÍNDICE DE FIGURAS, TABLAS Y FRAGMENTOS DE CÓDIGO

Figura 1. Arquitectura general del sistema Aether, de sensores a dashboard.

Figura 2. Portada real del dashboard (`index.html`), corrida local del 1 de septiembre de 2026.

Figura 3. Franja de módulos de la portada, con conteo de Declared Inventory en vivo.

Figura 4. Tabla de Declared Inventory (`inventory.html`) con datos reales.

Figura 5. Panel de bitácora de inspecciones, una corrida expandida.

Figura 6. Línea de manufactura – enlace punto a punto entre estaciones (diagrama).

Figura 7. Sección Línea de manufactura de la portada – estaciones pendientes y Visión con estado real.

Tabla 1. Decisiones arquitectónicas de Fase 0 (ADR 0001 a ADR 0008).

Tabla 2. Componentes del sistema y estado de implementación.

Tabla 3. Diagnóstico del problema por categoría.

Tabla 4. Alternativas de solución evaluadas y descartadas en Fase 0.

Tabla 5. Endpoints reales del backend (FastAPI).

Código 1. Árbol real del repositorio, generado al momento de este reporte.

Código 2. `test_combined.py` – bucle real de detección combinada (YOLO + código + ArUco).

Código 3. `aggregate_observations.py` – función `aggregate()` completa.

Código 4. `backend/main.py` – endpoint `retire_inventory_entry()`.

Código 5. `dashboard/app.js` – función `renderDelta()`.

Código 6. backend/main.py – confirmación binaria de la estación de Visión.

INTRODUCCIÓN

Este reporte documenta el desarrollo del proyecto integrador Aether, un sistema de manufactura automotriz que combina una unidad móvil de inspección de inventario con dos brazos robóticos industriales y un sistema de visión de verificación. El proyecto se desarrolla en el marco de la materia integradora de la Universidad Tecnológica de Chihuahua Sur, con un equipo de seis integrantes distribuidos por especialidad: programación de robots y visión computacional, diseño mecánico, programación del brazo KUKA, programación del brazo UR, mecánica y electrónica, e interfaz gráfica.

El alcance del proyecto no se limita a un robot móvil de inventario. El objetivo académico obligatorio es una línea de manufactura completa: un brazo KUKA KR6 coloca piezas automovilísticas en una banda transportadora, un brazo UR5 las ensambla mediante trayectorias pre-programadas, un sistema de visión confirma de forma binaria que el ensamble se completó correctamente y la unidad móvil Aether Inventory recoge la pieza terminada, la transporta y mantiene el control del inventario de todo el proceso.

Este documento presenta el estado real del desarrollo al momento de su redacción. Los módulos de percepción, procesamiento de inventario, backend y dashboard cuentan con implementación funcional verificada sobre hardware real. Los módulos de manipulación robótica, navegación autónoma y línea de manufactura física dependen de hardware que el equipo aún no tiene instalado, y se documentan en su estado de diseño arquitectónico, marcados explícitamente como pendientes donde corresponde.

Esta distinción entre lo construido y lo planeado se mantiene de forma consistente en todo el documento porque es un principio de trabajo del propio proyecto: no se reportan datos ni resultados que no provienen de una prueba real sobre hardware real.

1 CAPÍTULO 1. LA IDEA DEL PROYECTO

1.1 Contexto

Aether se desarrolla en un contexto de manufactura automotriz, industria con presencia significativa en Chihuahua y con una demanda constante de líneas de producción que combinen manipulación robótica, verificación de calidad y control de inventario. El proyecto integra tres frentes que normalmente se resuelven por separado: la colocación y el ensamble de piezas mediante brazos robóticos, la verificación visual de que el ensamble se completó correctamente y el seguimiento del inventario de piezas terminadas mediante una unidad móvil autónoma.

1.2 Antecedentes del problema

En líneas de manufactura pequeñas y en entornos académicos, la coordinación entre estaciones de trabajo suele depender de operadores humanos que trasladan piezas entre procesos, verifican visualmente el resultado de cada operación y llevan el conteo de inventario de forma manual. Esta dependencia introduce tres problemas recurrentes: tiempos muertos entre estaciones mientras se espera la intervención humana, inconsistencia en la verificación de calidad porque el criterio de inspección varía entre personas y momentos del día, y registros de inventario que no reflejan con precisión la producción real porque el conteo manual acumula errores conforme aumenta el volumen.

El proyecto responde a estos tres problemas con una solución integrada: brazos robóticos para la manipulación, un sistema de visión para la verificación de calidad y una unidad móvil autónoma para el seguimiento del inventario, comunicados entre sí sin depender de intervención humana en el flujo normal de operación.

1.3 Objetivo del proyecto

Desarrollar una línea de manufactura automotriz funcional que integre dos brazos robóticos industriales, un sistema de visión artificial de verificación y una unidad móvil de inventario, capaz de ejecutar el ciclo completo de colocación, ensamble, verificación y transporte de piezas terminadas sin orquestación central, y de mantener un registro trazable del inventario resultante.

Objetivos específicos

- Implementar un sistema de percepción híbrido capaz de detectar y contar objetos mediante YOLO, e identificarlos de forma exacta mediante lectura de código de barras o QR cuando esté disponible.
- Implementar un sistema de localización basado en marcadores ArUco para verificación de posición, sin depender de este sistema para la navegación del robot.
- Programar los brazos KUKA KR6 y UR5 mediante sus lenguajes nativos de fabricante, con trayectorias pre-programadas para la tarea de ensamble.
- Implementar un sistema de visión de línea que confirme de forma binaria si una pieza quedó correctamente ensamblada.
- Diseñar un esquema de datos que distinga entre observaciones crudas de percepción e inventario declarado, preservando la trazabilidad histórica de cada corrida de inspección.
- Integrar las cuatro estaciones del proceso mediante un protocolo de enlace punto a punto, sin un orquestador central que dirija el proceso.
- Construir un backend y un dashboard que muestren el inventario declarado, su historial y su trazabilidad de forma honesta, sin presentar como tiempo real información que no lo es.

Justificación

La automatización de líneas de manufactura pequeñas suele encontrarse con una disyuntiva: las soluciones comerciales completas superan el presupuesto de una pequeña o mediana empresa, mientras que las soluciones parciales dejan huecos de coordinación que terminan resolviéndose con intervención manual. Aether explora una tercera vía, con alcance académico: construir cada módulo con tecnología accesible y bien documentada — Python, OpenCV, YOLO, FastAPI, MQTT y programación nativa de fabricante para los brazos — e integrar los módulos mediante un protocolo de enlace simple en vez de un orquestador central costoso.

El valor del proyecto no está únicamente en la robótica de manipulación, que es la parte más visible de una línea de manufactura, sino en la capa de datos que la acompaña: un inventario que distingue entre lo observado y lo declarado, que nunca sobrescribe silenciosamente un resultado anterior con uno parcial y que mantiene una bitácora completa de cada corrida. Esta capa de datos es la que permite que la línea de manufactura sea auditable, y es una de las

contribuciones específicas del proyecto sobre un sistema de manipulación robótica convencional.

2 CAPÍTULO 2. DISEÑO DEL PROYECTO

2.1 Marco de decisiones de Fase 0

Antes de escribir la primera línea de implementación, el equipo cerró una etapa de decisiones de arquitectura llamada Fase 0: qué enfoque de percepción usar, cómo localizar al robot, cómo comunicar los módulos entre sí, y así sucesivamente. Cada una de estas decisiones se documenta como un ADR (Architecture Decision Record) numerado, archivado en docs/architecture/decisions/ del repositorio — un documento corto por decisión, con su contexto, la opción elegida y la justificación de por qué se descartaron las demás.

Las ocho decisiones de Fase 0 están cerradas y se citan por número en el resto de este reporte ("Decisión 3", "ADR 0008"); la Tabla 1 las resume para que esas referencias tengan un punto de consulta único.

Nº	DECISIÓN	RESOLUCIÓN
1	Percepción	Híbrido: detección/conteo por YOLO + identificación por código de barras/QR. Clasificación visual pura de SKU, diferida.
2	Localización	Marcadores ArUco para verificación de posición — nunca un sistema de navegación.
3	Navegación	Waypoints con odometría por encoders + detección reactiva de obstáculos (no "evasión inteligente").
4	Hardware de cómputo	Jetson Orin Nano Super 8GB (principal) / Raspberry Pi 5 + AI HAT+ (contingencia). Control de bajo nivel: ESP32 o STM32.
5	Esquema de datos	Observations (crudo) vs. Declared Inventory (declarado); estados PARTIAL / COMPLETE / INVALID.
6	Comunicaciones	MQTT para telemetría, REST para el dashboard. Parada de seguridad independiente de la conectividad de red.
7	Alcance del dashboard	Opción B: Inventario Declarado + Historial y Trazabilidad, solo lectura. Monitoreo en vivo, evolución obligatoria posterior al MVP.
8	Línea de manufactura	KUKA → UR5 → visión → Aether Inventory, handshake punto a punto (8a), programación nativa por fabricante (8b), visión binaria con el stack ya establecido (8c).

Tabla 1. Decisiones arquitectónicas de Fase 0 (ADR 0001 a ADR 0008).

Dos principios de trabajo, también citados por número más adelante, aplican a todo el proyecto sin excepción (aether_context_docs.md, sección 4): el Principio 4.1, Honestidad de producto — "el producto vende lo que el cliente realmente compra... nunca sobre-prometer

capacidades no verificadas" —, y el Principio 4.5, No fabricar dependencias, campos o estructuras especulativas — "no agregar librerías, campos de esquema, endpoints, o cualquier otro elemento técnico 'por si acaso' antes de que exista una decisión explícita que lo requiera". Ambos son la razón detrás de casi todos los avisos [PENDIENTE] y calloutPending de este documento: no se fabrica ninguna garantía ni ningún dato que el sistema todavía no pueda respaldar con hardware real.

2.2 Componentes del sistema

La Tabla 2 resume los componentes que integran Aether, su función dentro del sistema y su estado real de implementación al momento de este reporte.

COMPONENTE	FUNCIÓN	ESTADO
Robot móvil Aether Inventory	Recorre el entorno, detecta productos y transporta piezas terminadas a su ubicación de almacenamiento. Lleva montada la cámara de percepción.	Pendiente
Brazo KUKA KR6	Primera estación de la línea. Coloca piezas automovilísticas sobre la banda transportadora mediante programación nativa en KRL.	Pendiente
Brazo UR5	Segunda estación. Ensambla las piezas colocadas por el KUKA mediante trayectorias pre-programadas, sin aprendizaje por demostración.	Pendiente
Sistema de visión de línea	Confirma de forma binaria si el ensamble se completó correctamente, con el mismo stack de percepción del robot móvil.	Decidido
Cómputo de borde — Jetson Orin Nano Super	Ejecuta el pipeline de percepción del robot móvil. Contingencia definida: Raspberry Pi 5 con AI HAT+.	Pendiente
Control de bajo nivel — ESP32 / STM32	Motores, PWM por hardware y watchdog de seguridad. No ejecuta percepción: no cuenta con GPU.	Pendiente
Percepción híbrida	YOLO + código de barras/QR + ArUco, corriendo sobre cámara de laptop en modo demo.	Funcionando
Inventory Engine	Agrupar observaciones crudas en inventario declarado, con historial trazable.	Funcionando
Backend (FastAPI) y base de datos	Expone el inventario declarado, su historial y una operación de retiro protegida, respaldado por SQLite.	Funcionando
Dashboard	Interfaz web del inventario declarado y su bitácora, servida como archivos estáticos por el backend.	Funcionando

Tabla 2. Componentes del sistema y estado de implementación.

2.3 Diagnóstico

El diagnóstico del problema se organiza en cuatro categorías: personal, método, detección y entorno, resumidas en la Tabla 3.

CATEGORÍA	HALLAZGO / RIESGO
Personal	La dependencia de operadores humanos para trasladar piezas entre estaciones y para contar inventario introduce variabilidad: el criterio y la velocidad cambian de una persona a otra y a lo largo del turno.
Método	Sin un protocolo de enlace entre estaciones, cada operación queda aislada y el operador humano se convierte en el mecanismo de coordinación por defecto, lo que genera tiempos muertos entre procesos.
Detección	Con reflejo de luz sobre un marcador ArUco, el detector puede reportar un identificador incorrecto pero válido dentro del diccionario, sin señal de error visible — hallazgo verificado durante las pruebas del módulo de percepción.
Entorno	La variabilidad de iluminación y la falta de puntos fijos de referencia en un entorno de manufactura no controlado incrementan el riesgo de error en la detección de objetos y en la verificación de posición.

Tabla 3. Diagnóstico del problema por categoría.

Los hallazgos de la categoría de detección tienen una implicación directa para la instalación física de los marcadores reales: la mitigación considerada — exigir consistencia de lectura en varios fotogramas, o usar marcadores impresos mate en vez de pantallas — queda pendiente de aplicarse una vez que se defina la iluminación del entorno de manufactura real.

2.4 Alternativas de solución

Antes de fijar la arquitectura del proyecto se evaluaron y descartaron formalmente varias alternativas, documentadas como decisiones de la Fase 0 del proyecto y resumidas en la Tabla 4.

ALTERNATIVA DESCARTADA	DECISIÓN TOMADA	RAZÓN
Aprendizaje por demostración (LfD) para programar los brazos	Programación nativa de fabricante con trayectorias pre-programadas	El alcance académico no requiere generalizar tareas nuevas; la programación nativa da un comportamiento predecible y verificable, indispensable en una línea real.
ArUco como sistema de navegación completo	ArUco como capa de verificación de posición; navegación por waypoints y encoders	Un sistema de navegación basado solo en marcadores es frágil ante oclusiones y errores de lectura.

ALTERNATIVA DESCARTADA	DECISIÓN TOMADA	RAZÓN
SLAM para la navegación del robot móvil	Diferido como característica futura	Excede el tiempo disponible del proyecto (~240 horas en 4 meses) y no es indispensable para el ciclo completo en laboratorio.
Clasificación visual pura de producto, sin código	Diferida; se usa el enfoque híbrido YOLO + código de barras/QR	Requeriría un dataset de entrenamiento propio por tipo de pieza, esfuerzo no justificado frente al enfoque híbrido.
Orquestador central para las 4 estaciones	Protocolo de enlace punto a punto (I/O digital + MQTT)	Un orquestador central agrega un punto único de falla no justificado para una secuencia fija de 4 estaciones.

Tabla 4. Alternativas de solución evaluadas y descartadas en Fase 0.

3 CAPÍTULO 3. EJECUCIÓN DEL TRABAJO

3.1 Arquitectura general

La arquitectura del sistema se organiza como una tubería lineal desde el robot hasta el dashboard (Figura 1). Cada etapa se probó de forma independiente antes de integrarse con la siguiente, siguiendo el principio de desarrollo por módulos del proyecto: ningún módulo se integra con el siguiente sin haberse probado primero de forma aislada.

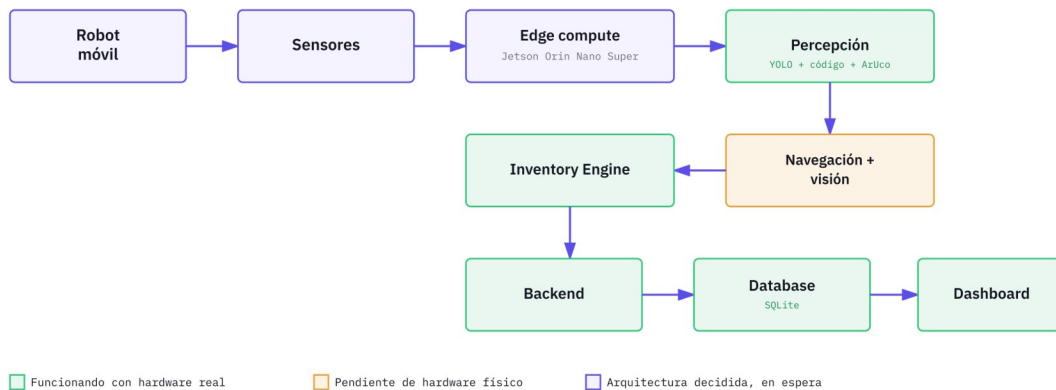


Figura 1. Arquitectura general del sistema Aether, de sensores a dashboard.

3.2 Estructura de carpetas

El repositorio es un monorepo diseñado, desde el día uno, para que el módulo de manipulación (línea de manufactura) tenga espacio de primera clase sin forzar su implementación temprana: las carpetas de manipulación, firmware y hardware existen aunque su hardware real todavía no llega, y varias siguen intencionalmente vacías — una carpeta vacía con un README explicando por qué no es lo mismo que una carpeta olvidada. El árbol de la Figura de abajo se generó recorriendo el repositorio real al momento de este reporte (excluyendo entornos virtuales, cachés y archivos `__init__.py` repetitivos, para que quepa en una página), no se escribió a mano.

```

INTEGRADORAHORASI/
|-- backend/
|   |-- api/
|   |   |-- .gitkeep
|   |-- db/
|   |   |-- migrations/
|   |   |   |-- .gitkeep

```

```
| | |-- models/
| | | |-- .gitkeep
| | | |-- declared_entry_history.py
| | | |-- declared_entry.py
| | | |-- database.py
| |-- logs/
| | |-- backend.log
| |-- scripts/
| | |-- load_from_edge.py
| |-- services/
| | |-- .gitkeep
| |-- aether.db
| |-- logging_config.py
| |-- main.py
| |-- pyproject.toml
| |-- README.md
| |-- requirements-dev.txt
| |-- requirements.txt
| |-- yolov8n.pt
|-- contracts/
| |-- inventory_to_manipulation.schema.json
| |-- line_handshake_protocol.md
| |-- mqtt_topics.md
| |-- rest_api.md
|-- dashboard/
| |-- .gitkeep
| |-- app.js
| |-- camera-stream.js
| |-- index.html
| |-- inventory.html
| |-- line-vision.js
| |-- node-field.js
| |-- portada.js
| |-- style.css
|-- docs/
| |-- academic/
| | |-- AI_CONTEXT.md
| | |-- arquitectura_ideal.md
| |-- architecture/
| | |-- decisions/
| | | |-- 0001-percepcion.md
| | | |-- 0002-localizacion.md
| | | |-- 0003-navegacion.md
| | | |-- 0004-hardware-computo.md
| | | |-- 0005-esquema-datos.md
| | | |-- 0006-comunicaciones.md
| | | |-- 0007-alcance-dashboard.md
| | | |-- 0008-integracion-manipulacion.md
| | |-- criterios_aceptacion.md
```

```

| | | L-- diagrams/
| | | | L-- .gitkeep
| | L-- quotations/
| | | L-- .gitkeep
| | edge/
| | | |-- comms/
| | | | |-- mqtt_client/
| | | | | L-- .gitkeep
| | | | L-- safety_independent_stop/
| | | | | L-- .gitkeep
| | | |-- inventory_engine/
| | | | |-- declared_inventory/
| | | | | |-- .gitkeep
| | | | | L-- declared_entry.py
| | | | |-- observations/
| | | | | |-- .gitkeep
| | | | | L-- observation.py
| | | | |-- aggregate_observations.py
| | | | |-- coverage_states.py
| | | | L-- declared_inventory_output.json
| | |-- logs/
| | | L-- edge.log
| | |-- manipulation/
| | | |-- arm_control/
| | | | |-- kuka_kr6/
| | | | | |-- .gitkeep
| | | | | L-- README.md
| | | | L-- ur5/
| | | | | |-- .gitkeep
| | | | | L-- README.md
| | | |-- lfd/
| | | | |-- demonstration_capture/
| | | | | L-- .gitkeep
| | | | |-- pose_estimation/
| | | | | L-- .gitkeep
| | | | |-- segmentation/
| | | | | L-- .gitkeep
| | | | | L-- README.md
| | | |-- line_events_listener/
| | | | L-- .gitkeep
| | | |-- line_vision/
| | | | L-- .gitkeep
| | | L-- task_planning/
| | | | L-- README.md
| | |-- navigation/
| | | |-- obstacle_detection/
| | | | L-- .gitkeep
| | | |-- odometry/
| | | | L-- .gitkeep

```

```

| |   |-- waypoints/
| |       |-- .gitkeep
| |   |-- perception/
| |       |-- aruco_localization/
| |           |-- .gitkeep
| |       |-- aruco_markers/
| |           |-- aruco_0.png
| |           |-- aruco_1.png
| |           |-- aruco_2.png
| |           |-- aruco_3.png
| |       |-- barcode_qr/
| |           |-- .gitkeep
| |       |-- yolo_detection/
| |           |-- .gitkeep
| |       |-- _capture_report_frame.py
| |       |-- _report_frame.png
| |       |-- generate_aruco_markers.py
| |       |-- observations_output.json
| |       |-- test_aruco.py
| |       |-- test_barcode.py
| |       |-- test_camera.py
| |       |-- test_combined.py
| |       |-- test_detection_rate.py
| |       |-- test_yolo.py
| |       |-- yolov8n.pt
| |   |-- logging_config.py
| |   |-- pyproject.toml
| |   |-- README.md
| |   |-- requirements-dev.txt
| |   |-- requirements.txt
|-- firmware/
|   |-- motor_control/
|       |-- .gitkeep
|   |-- pwm_drivers/
|       |-- .gitkeep
|   |-- safety_watchdog/
|       |-- .gitkeep
|-- hardware/
|   |-- bom/
|       |-- .gitkeep
|   |-- wiring/
|       |-- .gitkeep
|-- scripts/
|   |-- .gitkeep
|-- tests/
|   |-- backend/
|       |-- .gitkeep
|       |-- test_main.py
|       |-- test_setup.py

```

```

|   |-- edge/
|   |   |-- .gitkeep
|   |   |-- test_aggregate_observations.py
|   |   |-- test_setup.py
|   |   |-- manipulation/
|   |       |-- .gitkeep
|-- .gitignore
|-- aether_context_docs.md
|-- CLAUDE.md
|-- package-lock.json
|-- package.json
|-- PyWhatKit_DB.txt
|-- skills-lock.json
|-- yolov8n.pt

```

Código 1. Árbol real del repositorio (raíz, y subcarpetas de edge/ y backend/), generado al momento de este reporte.

backend/ y edge/ — por qué tienen entornos virtuales separados

edge/ y backend/ cada uno declara su propio .venv/ y su propio requirements.txt (edge/README.md, backend/README.md) porque corren en hardware distinto en producción real: edge/ es el software que eventualmente corre en el Jetson Orin Nano Super montado en el robot móvil (Decisión 4), con builds de PyTorch/OpenCV empaquetadas por NVIDIA JetPack; backend/ corre en un servidor o laptop convencional. Mezclar sus dependencias en un solo entorno arriesgaría asumir que un wheel compilado para esta laptop de desarrollo (CPU, Windows) sirve igual en el Jetson (ARM64, CUDA) sin haberlo verificado — exactamente el tipo de suposición no verificada que el proyecto evita (Principio 4.5). Por eso edge/requirements.txt instala torch aparte, apuntando al índice CPU oficial, con una nota explícita de no reusar ese mismo procedimiento en el Jetson sin comprobarlo.

Dentro de edge/, la carpeta perception/ es la única con lógica real hoy: los scripts test_camera.py, test_yolo.py, test_barcode.py y test_aruco.py verifican cada pieza del pipeline de forma aislada, test_combined.py es el pipeline combinado real (Código 2, sección 3.3), test_detection_rate.py mide tasa de detección, y generate_aruco_markers.py generó los PNG reales en aruco_markers/ que usa el equipo para las pruebas de localización. inventory_engine/ también es real (Código 3, sección 3.6). El resto de edge/ — comms/, navigation/, manipulation/ — son paquetes de Python ya andamiados (con su .gitkeep o su README) pero sin lógica todavía, a la espera de hardware: comms/mqtt_client/ y comms/safety_independent_stop/ esperan el microcontrolador de bajo nivel; navigation/waypoints/, odometry/ y obstacle_detection/ esperan el chasis, los motores y los encoders del robot móvil.

Dentro de `edge/manipulation/`, `arm_control/kuka_kr6/` y `arm_control/ur5/` están deliberadamente vacías salvo su README: por la sub-decisión 8b de ADR 0008, la trayectoria de cada brazo se programa directamente en su controlador nativo (KRL / URScript-PolyScope) y nunca vive en este repositorio — lo único que vivirá aquí es el disparo externo simple (I/O digital o puente MQTT→I/O) una vez que el equipo confirme qué interfaz de E/S exponen los brazos reales. `task_planning/` y `lfd/` (con sus subcarpetas `pose_estimation/`, `segmentation/` y `demonstration_capture/`) llevan cada una un README.md que explica que quedaron fuera de alcance del MVP por ADR 0008 — sin planeador de tareas propio ni Learning from Demonstration — y que reabrir esa decisión requeriría una ADR nueva, no una implementación directa ahí.

firmware/ y hardware/ — vacías a propósito, no basura sin limpiar

`firmware/motor_control/`, `firmware/pwm_drivers/` y `firmware/safety_watchdog/` solo contienen un `.gitkeep`: son el espacio reservado para el código del ESP32 o STM32 (Decisión 4) que hoy el equipo todavía no tiene instalado — control de bajo nivel, PWM por hardware y el watchdog de seguridad independiente de red (Decisión 6). De la misma forma, `hardware/bom/` y `hardware/wiring/` esperan el bill of materials y los diagramas de cableado reales, que solo tienen sentido una vez comprado el hardware. Ambas carpetas existen ya en el repositorio, en vez de crearse el día que llegue el hardware, siguiendo el mismo principio que `manipulation/`: reservar el espacio desde el diseño de Fase 0 sin fabricar contenido que todavía no existe.

scripts/ (raíz) vs. backend/scripts/

`scripts/` en la raíz del repositorio está reservada para utilidades que cruzan todo el monorepo — `setup`, despliegue, tareas que tocarían `edge/`, `backend/` y `dashboard/` a la vez — y hoy solo tiene un `.gitkeep` porque esa necesidad todavía no existe. `backend/scripts/load_from_edge.py` es distinta a propósito: es un script que solo le concierne a `backend/`, porque solo toca su base de datos (lee `edge/inventory_engine/declared_inventory_output.json` y hace `upsert` en `aether.db`, más una fila nueva en la bitácora — ver Código 3 y 3.6). No subir `load_from_edge.py` a la raíz evita sugerir que es una utilidad general del proyecto cuando en realidad es específica del backend.

tests/ — por qué manipulation/ está vacía

`tests/backend/` y `tests/edge/` ya tienen pruebas reales (`test_main.py` con siete pruebas sobre los cuatro endpoints reales del backend, y `test_aggregate_observations.py` sobre la función `aggregate()` del Inventory Engine). `tests/manipulation/` sigue con solo un `.gitkeep`, no por descuido, sino porque todavía no hay ninguna lógica de manipulación que probar: por ADR

0008, lo único que eventualmente vivirá en `edge/manipulation/` es el disparo I/O hacia cada brazo y el listener MQTT de `line_events_listener/`, ninguno de los cuales existe todavía. La carpeta de pruebas se reservó desde el diseño de la estructura (`aether_context_docs.md`, sección 6) para que, cuando esa lógica exista, ya tenga un lugar designado — la misma filosofía de "espacio reservado desde el día uno" que aplica a `edge/manipulation/` en general.

3.3 Percepción

El módulo de percepción implementa la Decisión 1 del proyecto: un enfoque híbrido que combina YOLO para la detección y el conteo de objetos con lectura de código de barras o QR para la identidad exacta del producto. El script `test_combined.py` ejecuta, sobre cada fotograma, tres pasos en orden: detección de marcadores ArUco sobre el fotograma completo, detección de objetos mediante YOLO y, dentro de cada caja delimitadora detectada, un intento de lectura de código mediante `pyzbar`, acotado a esa región para que el código quede asociado al objeto correcto.

Cada detección se dibuja con su caja y su clase, tenga o no un código legible asociado, porque la Decisión 1 establece que ninguna detección se descarta. Cada detección genera un registro `Observation`, sin deduplicar entre fotogramas, con los campos: marca de tiempo, clase detectada, dato del código si se leyó, bandera de lectura, confianza de YOLO e identificador del marcador ArUco visible en ese fotograma. En una corrida de prueba real, el pipeline generó 712 `Observations` en pocos minutos de operación manual.

El Código 2 muestra el núcleo real del bucle de detección: por cada caja de YOLO se recorta la región, `pyzbar` intenta leer un código dentro de ese recorte, y el resultado — con o sin código — se empaqueta como una `Observation` junto con el marcador ArUco visible en ese mismo fotograma.

```
aruco_corners, aruco_ids, _ = aruco_detector.detectMarkers(frame)
location_marker_id = int(aruco_ids.flatten()[0]) if aruco_ids is not None else None
if aruco_ids is not None:
    cv2.aruco.drawDetectedMarkers(frame, aruco_corners, aruco_ids)

results = model(frame, verbose=False)

for box in results[0].boxes:
    x1, y1, x2, y2 = (int(v) for v in box.xyxy[0])
    class_name = model.names[int(box.cls[0])]
    confidence = float(box.conf[0])
```

```

# Recorte de la caja delimitadora: pyzbar solo busca código
# dentro de la región de este objeto, no en el frame entero.
crop = frame[y1:y2, x1:x2]
codes = decode(crop) if crop.size > 0 else []
data = codes[0].data.decode("utf-8") if codes else None

cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
# ... cv2.putText de clase y código (omitido aquí, ver archivo real) ...

observations.append(
    Observation(
        timestamp=datetime.now(UTC).isoformat(),
        detected_class=class_name,
        code_data=data,
        code_read=bool(codes),
        confidence=confidence,
        location_marker_id=location_marker_id,
    )
)

```

Código 2. edge/perception/test_combined.py, líneas 104-173 (abreviado) – bucle real de detección combinada.

PENDIENTE — el volumen de Observations crudas no tiene todavía una política de retención ni muestreo; suficiente para verificación puntual, pendiente de definir antes de operación continua real.

PENDIENTE — figura pendiente — un frame real de test_combined.py (o de la captura equivalente sin interfaz gráfica) mostrando YOLO + ArUco + lectura de código a la vez. Se agregará como Figura numerada, con figure(), en cuanto el equipo capture esa evidencia con la cámara física conectada — ver README_CLAUDE_CODE.md sobre cómo agregarla sin dejar este aviso junto a la evidencia real.

3.4 Localización

Los marcadores ArUco se usan exclusivamente como capa de verificación de posición (Decisión 2), nunca como sistema de navegación. El primer marcador detectado en cada fotograma se asigna como location_marker_id de todas las Observations generadas en ese fotograma.

Durante las pruebas del módulo se identificó una limitación real del enfoque: con el marcador mostrado en pantalla de celular y reflejo de luz sobre la imagen, el detector reportó

identificadores incorrectos pero válidos dentro del diccionario en lugar de fallar de forma explícita, comportamiento esperado de la corrección de errores integrada en ArUco. Esta limitación se documenta como pendiente de mitigación una vez que se definan la iluminación y la ubicación física de los marcadores reales.

3.5 Navegación

PENDIENTE — depende del chasis, los motores y los encoders físicos del robot móvil, que el equipo aún no tiene instalados.

La Decisión 3 del proyecto establece navegación por waypoints con odometría por encoders y detección reactiva de obstáculos, sin evasión inteligente ni replanificación de rutas. El contrato de estados de cobertura (PARTIAL, COMPLETE, INVALID) que depende de este módulo ya está definido en `coverage_states.py`, sin usarse todavía — se dejó pendiente a propósito en vez de fabricar esa lógica sin una base de navegación real.

3.6 Inventory Engine

El módulo de inventario separa dos capas de datos siguiendo la Decisión 5 del proyecto: `Observations`, el registro crudo de cada detección individual, y `Declared Inventory`, el resumen procesado que agrupa observaciones en entradas de inventario. La función `aggregate()` recibe una lista de `Observations` y agrupa por código de producto exacto cuando existe, o por el prefijo `unidentified:` seguido de la clase detectada cuando no hay código legible.

Por cada grupo calcula la clase más frecuente, el total de observaciones, cuántas tenían código legible, la confianza promedio, el rango temporal de primera y última detección y el conteo de observaciones por ubicación. En una corrida real de prueba, 712 `Observations` se agruparon en 19 entradas de `DeclaredEntry`. Seis pruebas automatizadas verifican esta función: agrupación por código, prefijo `unidentified:`, promedio de confianza, orden correcto de primera y última detección con datos desordenados, conteo de lecturas de código y agrupación por ubicación.

El Código 3 es la función completa tal como vive en el repositorio: pura, sin efectos secundarios (no lee ni escribe archivos), lo que es precisamente lo que permite probarla con las seis pruebas automatizadas mencionadas arriba sin necesitar cámara ni archivos de por medio.

```
def aggregate(observations: list[dict]) -> list[DeclaredEntry]:
```

```

"""Agrupar Observations crudas en DeclaredEntry – función pura.

No lee ni escribe archivos: recibe una lista de Observations (como
dicts, con las mismas claves que produce edge/perception/test_combined.py)
y regresa la lista de DeclaredEntry resultante, ordenada por identifier.
"""

groups: dict[str, list[dict]] = {}
for observation in observations:
    groups.setdefault(_group_key(observation), []).append(observation)

entries = []
for identifier, group in sorted(groups.items()):
    class_counts = Counter(o["detected_class"] for o in group)
    most_common_class = class_counts.most_common(1)[0][0]
    timestamps = sorted(o["timestamp"] for o in group)
    location_counts = Counter(_location_key(o) for o in group)

    entries.append(
        DeclaredEntry(
            identifier=identifier,
            detected_class=most_common_class,
            total_observations=len(group),
            code_read_count=sum(1 for o in group if o["code_read"]),
            avg_confidence=sum(o["confidence"] for o in group) / len(group),
            first_seen=timestamps[0],
            last_seen=timestamps[-1],
            location_counts=dict(location_counts),
        )
    )

return entries

```

Código 3. edge/inventory_engine/aggregate_observations.py, función aggregate() completa.

Una regla explícita del esquema evita que una inspección parcial sobrescriba silenciosamente un inventario declarado previo: el estado de cobertura de cada corrida queda pendiente de implementación hasta que el módulo de Navegación pueda reportar cobertura real.

3.7 Backend y comunicaciones

El backend, construido en FastAPI (Ramírez, s.f.), expone cuatro rutas reales, resumidas en la Tabla 5.

MÉTODO	RUTA	DESCRIPCIÓN	RESPUESTA
GET	/health	Verificación de estado del servicio.	{"status": "ok"}
GET	/inventory	Inventario declarado; filtra status="active" por defecto (include_retired=true para ver todo).	Lista de DeclaredEntry
PATCH	/inventory/{identifier}/retire	Cambia el estado a "retired". Nunca borra la fila. Requiere password contra ADMIN_ACTION_PASSWORD.	200 / 403 / 404
GET	/inventory/history	Bitácora completa, ordenada por fecha de registro descendente.	Lista de history rows

Tabla 5. Endpoints reales del backend (FastAPI).

La ruta de retiro (Código 4) es la única operación de escritura que expone el backend, y es también la que mejor ilustra el Principio 4.1 del proyecto (no fabricar una garantía de cobertura que el sistema todavía no puede respaldar): retirar una entrada nunca ocurre de forma automática por ausencia en una corrida de carga, solo por esta llamada explícita, hecha por una persona — y ni siquiera entonces borra la fila, solo cambia su estado.

```
@app.patch("/inventory/{identifier}/retire")
def retire_inventory_entry(identifier: str, body: RetireRequest):
    """Marca una entrada de Declared Inventory como "retired".

    [...] Requiere esta llamada explícita, hecha por una persona. Nunca
    ocurre automáticamente por ausencia en una corrida de load_from_edge.py.

    Retirar es un cambio de estado, no un borrado: la fila sigue existiendo
    y sigue siendo consultable vía GET /inventory?include_retired=true — el
    historial nunca se pierde.
    """
    if body.password != ADMIN_ACTION_PASSWORD:
        raise HTTPException(status_code=403, detail="Contraseña incorrecta.")

    session = SessionLocal()
    try:
        entry = session.get(DeclaredEntry, identifier)
        if entry is None:
            raise HTTPException(
                status_code=404,
                detail=f"No existe Declared Inventory con identifier
                '{identifier}'."
            )
        entry.status = "retired"
```

```

session.commit()
return {"identifier": entry.identifier, "status": entry.status}
finally:
    session.close()

```

Código 4. backend/main.py, función `retire_inventory_entry()` – la única escritura real que expone el backend.

La ruta de retiro está protegida con una contraseña de entorno que el propio proyecto documenta explícitamente como traba anti-clics-accidentales y no como autenticación real, sin usuarios, sesiones ni roles. La comunicación de telemetría del robot está definida para MQTT (Banks et al., 2019) y la del dashboard para REST (Decisión 6), con el comando de parada de seguridad diseñado para ser independiente de la conectividad de red, resuelto por un watchdog de hardware en el microcontrolador.

3.8 Dashboard

El dashboard se construyó sin frameworks ni herramientas de compilación, con HTML, CSS y JavaScript servidos como archivos estáticos por el propio backend. Sigue una identidad visual definida como "industrial dark tech": fondo casi negro, tipografía condensada reservada para títulos de sección, tipografía de cuerpo separada del resto de la interfaz y tipografía monoespaciada reservada para anotación técnica y cifras.

El color se usa con significado fijo: acento azul-violeta para lo navegable, verde únicamente para lo verificado con hardware real y ámbar para lo pendiente o lo que requiere atención. Las Figuras 3 y 4 muestran la portada real corriendo en esta máquina: el hero con la llamada a acción hacia el sistema, y la franja de módulos con el contador de Declared Inventory obtenido en vivo de GET /inventory — el pie de página de la propia portada aclara que ningún número ahí está escrito a mano (dashboard/portada.js).



Figura 3. Portada real del dashboard (index.html), corrida local del 1 de septiembre de 2026.

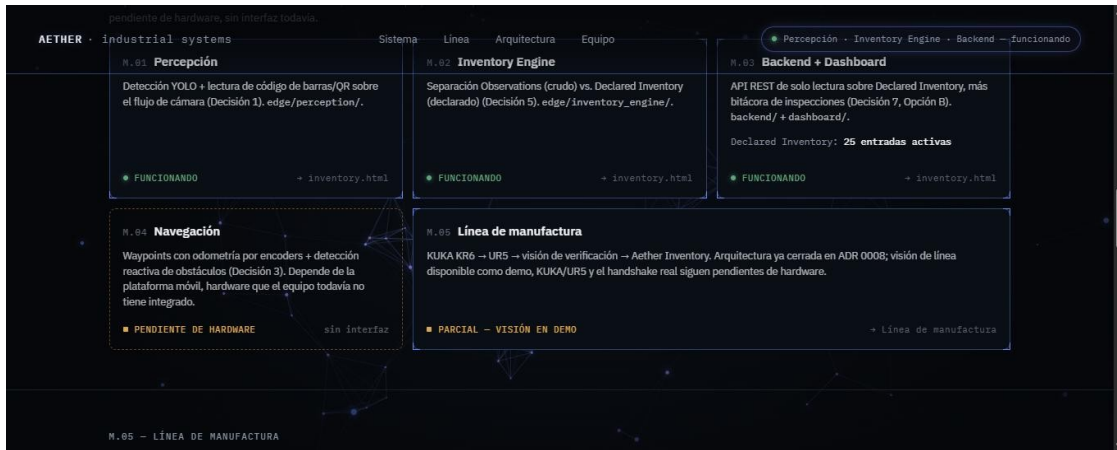


Figura 4. Franja de módulos de la portada, con "25 entradas activas" obtenido en vivo de GET /inventory.

PENDIENTE — figura pendiente — la sección Percepción de la portada (#percepcion) con el stream MJPEG activo mostrando una detección real (caja de YOLO + clase, y el código leído si lo hay). Se agregará como Figura numerada en cuanto se capture con la cámara física conectada.

El Código 5 es la lógica real detrás de un detalle honesto de la bitácora (Figura 6): cada corrida muestra cuánto cambió cada identifier contra la corrida anterior — "nuevo", "sin cambio" o un delta con signo — calculado siempre a partir de lo que ya devolvió la API, nunca de un valor asumido en el cliente.

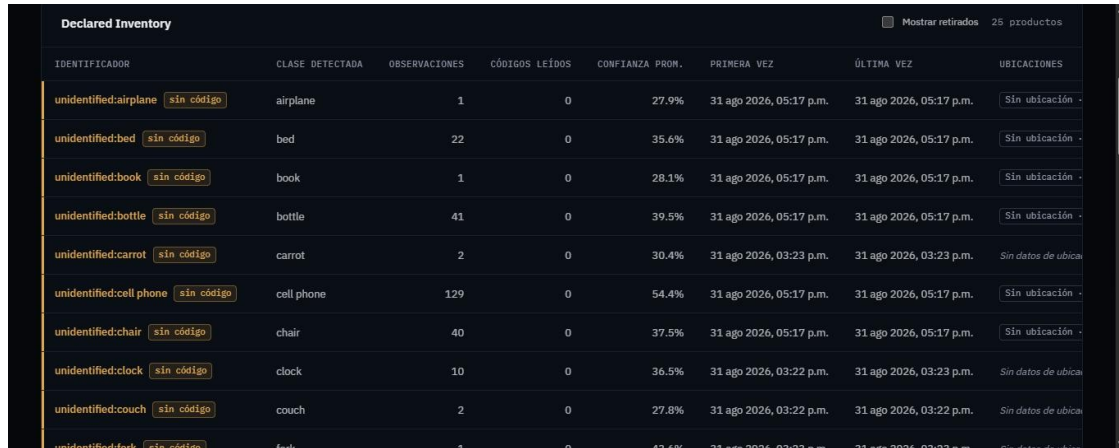
```
function renderDelta(current, previousRun) {
  // previousRun: null si no hay corrida anterior (nada que comparar); Map
  // (identifier -> fila) de la corrida anterior en caso contrario — puede
  // no incluir este identifier si es nuevo en esta corrida.
  if (previousRun === null) return "";

  const previousRow = previousRun.get(current.identifier);
  if (!previousRow) return '<span class="delta delta--new">nuevo</span>';

  const diff = current.total_observations - previousRow.total_observations;
  if (diff === 0) return '<span class="delta delta--flat">sin cambio</span>';
  const sign = diff > 0 ? "+" : "";
  return `<span class="delta delta--changed">${sign}${diff}</span>`;
}
```

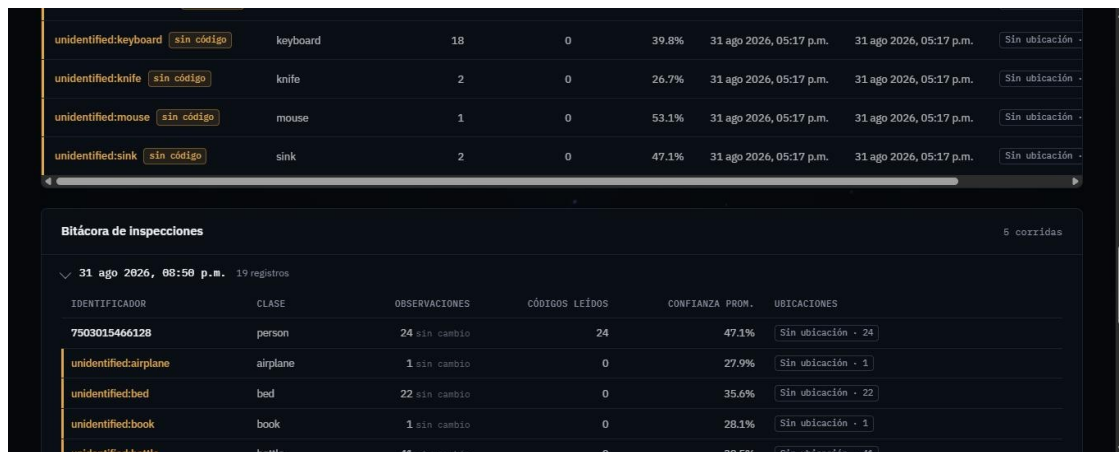
Código 5. dashboard/app.js, función renderDelta() — delta real entre corridas de la bitácora.

La página de inventario (Figura 5) muestra el inventario declarado con sus chips de ubicación y una operación de retiro protegida por contraseña que nunca borra el registro subyacente; el panel de bitácora (Figura 6) agrupa el historial por corrida, con la corrida más reciente expandida por defecto.



IDENTIFICADOR	CLASE DETECTADA	OBSERVACIONES	CÓDIGOS LEÍDOS	CONFIANZA PROM.	PRIMERA VEZ	ÚLTIMA VEZ	UBICACIONES
unidentified:airplane sin código	airplane	1	0	27.9%	31 ago 2026, 05:17 p.m.	31 ago 2026, 05:17 p.m.	Sin ubicación
unidentified:bed sin código	bed	22	0	35.6%	31 ago 2026, 05:17 p.m.	31 ago 2026, 05:17 p.m.	Sin ubicación
unidentified:book sin código	book	1	0	28.1%	31 ago 2026, 05:17 p.m.	31 ago 2026, 05:17 p.m.	Sin ubicación
unidentified:bottle sin código	bottle	41	0	39.5%	31 ago 2026, 05:17 p.m.	31 ago 2026, 05:17 p.m.	Sin ubicación
unidentified:carrot sin código	carrot	2	0	30.4%	31 ago 2026, 03:23 p.m.	31 ago 2026, 03:23 p.m.	Sin datos de ubica
unidentified:cell phone sin código	cell phone	129	0	54.4%	31 ago 2026, 05:17 p.m.	31 ago 2026, 05:17 p.m.	Sin ubicación
unidentified:chair sin código	chair	40	0	37.5%	31 ago 2026, 05:17 p.m.	31 ago 2026, 05:17 p.m.	Sin ubicación
unidentified:clock sin código	clock	10	0	36.5%	31 ago 2026, 03:22 p.m.	31 ago 2026, 03:23 p.m.	Sin datos de ubica
unidentified:couch sin código	couch	2	0	27.8%	31 ago 2026, 03:22 p.m.	31 ago 2026, 03:22 p.m.	Sin datos de ubica
unidentified:fork sin código	fork	1	0	43.6%	31 ago 2026, 03:23 p.m.	31 ago 2026, 03:23 p.m.	Sin datos de ubica

Figura 5. Tabla de Declared Inventory (inventory.html) con datos reales de una corrida de test_combined.py.



IDENTIFICADOR	CLASE	OBSERVACIONES	CÓDIGOS LEÍDOS	CONFIANZA PROM.	UBICACIONES
unidentified:keyboard sin código	keyboard	18	0	39.8%	31 ago 2026, 05:17 p.m.
unidentified:knife sin código	knife	2	0	26.7%	31 ago 2026, 05:17 p.m.
unidentified:mouse sin código	mouse	1	0	53.1%	31 ago 2026, 05:17 p.m.
unidentified:sink sin código	sink	2	0	47.1%	31 ago 2026, 05:17 p.m.

IDENTIFICADOR	CLASE	OBSERVACIONES	CÓDIGOS LEÍDOS	CONFIANZA PROM.	UBICACIONES
7503015466128	person	24 sin cambio	24	47.1%	Sin ubicación - 24
unidentified:airplane sin código	airplane	1 sin cambio	0	27.9%	Sin ubicación - 1
unidentified:bed sin código	bed	22 sin cambio	0	35.6%	Sin ubicación - 22
unidentified:book sin código	book	1 sin cambio	0	28.1%	Sin ubicación - 1
unidentified:bottle sin código	bottle	41 sin cambio	0	39.5%	Sin ubicación - 41

Figura 6. Panel de bitácora de inspecciones, corrida del 31 de agosto de 2026 expandida.

3.9 Línea de manufactura

La Decisión 8 del proyecto define la integración de las cuatro estaciones de la línea de manufactura mediante un enlace punto a punto entre estaciones (Figura 8), sin orquestador central: el KUKA KR6 coloca la pieza en la banda, el UR5 la ensambla, la visión de línea confirma el resultado de forma binaria y Aether Inventory recoge la pieza terminada. Un listener MQTT registra cada evento como Observation, sin dirigir ninguna parte del proceso.

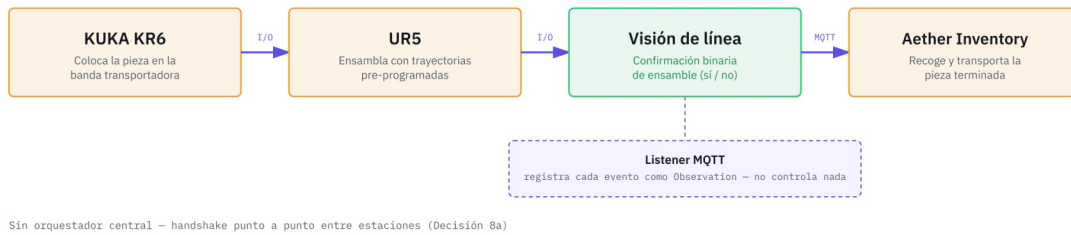


Figura 8. Línea de manufactura – enlace punto a punto entre estaciones.

El control de cada brazo usa programación nativa de fabricante — KRL para el KUKA, URScript o PolyScope para el UR5 — con disparo externo simple por entrada y salida digital o por un puente MQTT hacia I/O.

De las cuatro estaciones, hoy solo Visión tiene una demo real, reutilizando el mismo stack de percepción ya establecido (YOLO) — consistente con la propia sub-decisión 8c, que exige no introducir una tecnología de visión nueva solo para esta estación. El Código 6 es esa confirmación binaria completa: sin código de barras ni ArUco, solo "PIEZA DETECTADA" o "SIN PIEZA" según si YOLO reportó al menos una caja en el fotograma.

```

def _mjpeg_line_vision_frames(cap, model):
    """Generador MJPEG de confirmación binaria – mucho más simple que
    _mjpeg_frames: no lee código de barras ni ArUco, solo dice si hay o no
    algún objeto detectado en el frame ("PIEZA DETECTADA" / "SIN PIEZA")."""
    import cv2

    try:
        while True:
            ret, frame = cap.read()
            if not ret:
                break

            results = model(frame, verbose=False)
            piece_detected = len(results[0].boxes) > 0
            label = "PIEZA DETECTADA" if piece_detected else "SIN PIEZA"
            color = (0, 200, 0) if piece_detected else (0, 140, 255) # BGR

            cv2.putText(frame, label, (24, 48), cv2.FONT_HERSHEY_SIMPLEX, 1.1,
                color, 3)

            ok, buffer = cv2.imencode(".jpg", frame)
            if not ok:
                continue
            yield (b"--frame\r\nContent-Type: image/jpeg\r\n\r\n" +

```

```
buffer.tobytes() + b"\r\n")
finally:
    cap.release()
```

Código 6. backend/main.py, función `_mjpeg_line_vision_frames()` – confirmación binaria real de la estación de Visión.

La Figura 9 muestra la sección de Línea de manufactura de la portada tal como está hoy: honesta sobre las tres estaciones que dependen de hardware que el equipo todavía no tiene (KUKA KR6, UR5, Aether Inventory, cada una marcada "Pendiente de hardware"), y la estación de Visión mostrando su estado real — inactiva hasta que alguien presiona "Iniciar visión de línea", sin simular que las otras tres ya funcionan.

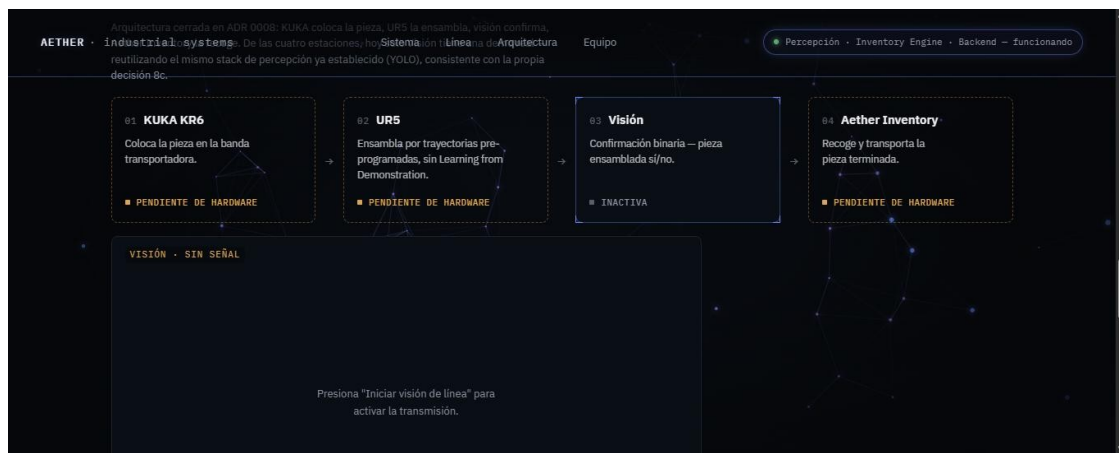


Figura 9. Sección Línea de manufactura de la portada – tres estaciones pendientes de hardware, Visión con su estado real.

PENDIENTE — la programación de los brazos KUKA y UR5, así como la instalación física de la línea de manufactura, dependen del hardware real de los brazos, que el equipo aún no tiene disponible. La interfaz de entrada y salida que exponen físicamente el KUKA KR6 y el UR5 para el disparo del enlace queda como investigación técnica pendiente, no como decisión de arquitectura abierta.

4 CAPÍTULO 4. ENTORNO VIRTUAL / FÍSICO Y MANTENIMIENTO

4.1 Entorno de desarrollo actual

El proyecto se desarrolla en dos entornos virtuales de Python independientes, uno para el subsistema de borde (edge/) y otro para el backend (backend/), cada uno con sus propias dependencias de producción y de desarrollo separadas. El control de versiones se lleva en un repositorio privado de GitHub.

El modo de operación actual es un modo de demostración de escritorio: una sola laptop ejecuta tanto la cámara de percepción como la cámara de visión de línea, controladas por variables de entorno independientes (`PERCEPTION_CAMERA_INDEX` y `LINE_VISION_CAMERA_INDEX`) porque ambos procesos comparten temporalmente el mismo equipo. Esta separación deja de ser necesaria de forma natural cuando el hardware físico esté instalado y cada proceso corra en una máquina distinta, y no se considera deuda técnica.

4.2 Entorno físico planeado

PENDIENTE — hardware no instalado al momento de este reporte.

Cuando el Jetson Orin Nano Super esté disponible (NVIDIA Corporation, 2024), la visión del robot móvil correrá sobre ese dispositivo, mientras que la visión de línea de manufactura correrá en una computadora fija de escritorio colocada cerca de la celda física, no en un dispositivo embebido: un microcontrolador ESP32 o STM32 no cuenta con GPU y no puede ejecutar YOLO, por lo que su función se limita al control de bajo nivel.

Queda pendiente instalar PyTorch con la compilación específica de NVIDIA JetPack para el Jetson, distinta de la compilación de CPU usada actualmente en la laptop de desarrollo.

4.3 Mantenimiento

PENDIENTE — no aplica todavía; depende de la instalación física del hardware.

Una vez instalado el hardware, el plan de mantenimiento deberá cubrir al menos la verificación periódica de la cámara y los marcadores de referencia, la calibración de los encoders y el

mantenimiento programado de los brazos KUKA y UR5 según la especificación de cada fabricante.

4.4 Manual de seguridad

El proyecto define un principio de seguridad desde la Decisión 6: el comando de parada de seguridad no depende de la conectividad de red, sino de un watchdog de hardware implementado en el microcontrolador de bajo nivel, de forma que una falla de comunicación nunca deje al robot sin capacidad de detenerse.

PENDIENTE — el resto del manual de seguridad — procedimientos de instalación, señalización del área de trabajo y protocolo de acceso de personal a la celda de manufactura — queda pendiente de definición hasta que el equipo cuente con los brazos robóticos y el robot móvil físicos. Documentar procedimientos de seguridad sin el hardware real presente sería una descripción especulativa que el proyecto evita de forma deliberada.

CONCLUSIONES Y RECOMENDACIONES

Al momento de este reporte, Aether tiene un flujo de datos completo y funcional que corre sobre hardware real desde la cámara hasta el dashboard: percepción híbrida con YOLO, código de barras y ArUco, un motor de inventario que separa observaciones crudas de inventario declarado, un backend en FastAPI con retiro trazable y una interfaz de "industrial dark tech" coherente en todo el sistema.

Esta base de software resultó más rápida de construir que la integración física de la línea de manufactura, lo cual es consistente con el orden de dependencias del proyecto: la lógica de datos no depende de hardware que aún no llega, mientras que la manipulación robótica y la navegación sí.

Los pendientes reales identificados en este reporte — programación de los brazos KUKA y UR5, navegación física del robot móvil, línea de manufactura instalada, y los umbrales de aceptación que dependen de hardware final — definen la ruta crítica restante del proyecto. Se recomienda priorizar la llegada del Jetson Orin Nano Super y de los brazos robóticos como siguiente hito, porque ambos desbloquean directamente los módulos de Navegación y de Manipulación, actualmente los dos con mayor dependencia externa.

Se recomienda también mantener el principio de trabajo que ha guiado el proyecto hasta ahora: no reportar como funcional ni como dato real nada que no provenga de una prueba verificada sobre hardware real. Este principio, aplicado en la separación entre observaciones y declarado, en los estados de cobertura pendientes y en las secciones marcadas explícitamente como pendientes en este mismo reporte, es lo que permite que el sistema sea auditable frente a un cliente o a un jurado académico.

REFERENCIAS

Banks, A., Briggs, E., Borgendale, K., & Gupta, R. (Eds.). (2019). MQTT Version 5.0 (OASIS Standard). OASIS. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>

Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLOv8 (Version 8.0.0) [Software]. GitHub. <https://github.com/ultralytics/ultralytics>

NVIDIA Corporation. (2024). Jetson Orin Nano Super Developer Kit [Especificaciones de producto]. <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/nano-super-developer-kit/>

OpenCV Team. (s.f.). Detection of ArUco markers. OpenCV Documentation. Recuperado en 2026, de https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html

Ramírez, S. (s.f.). FastAPI [Software]. Recuperado en 2026, de <https://fastapi.tiangolo.com>